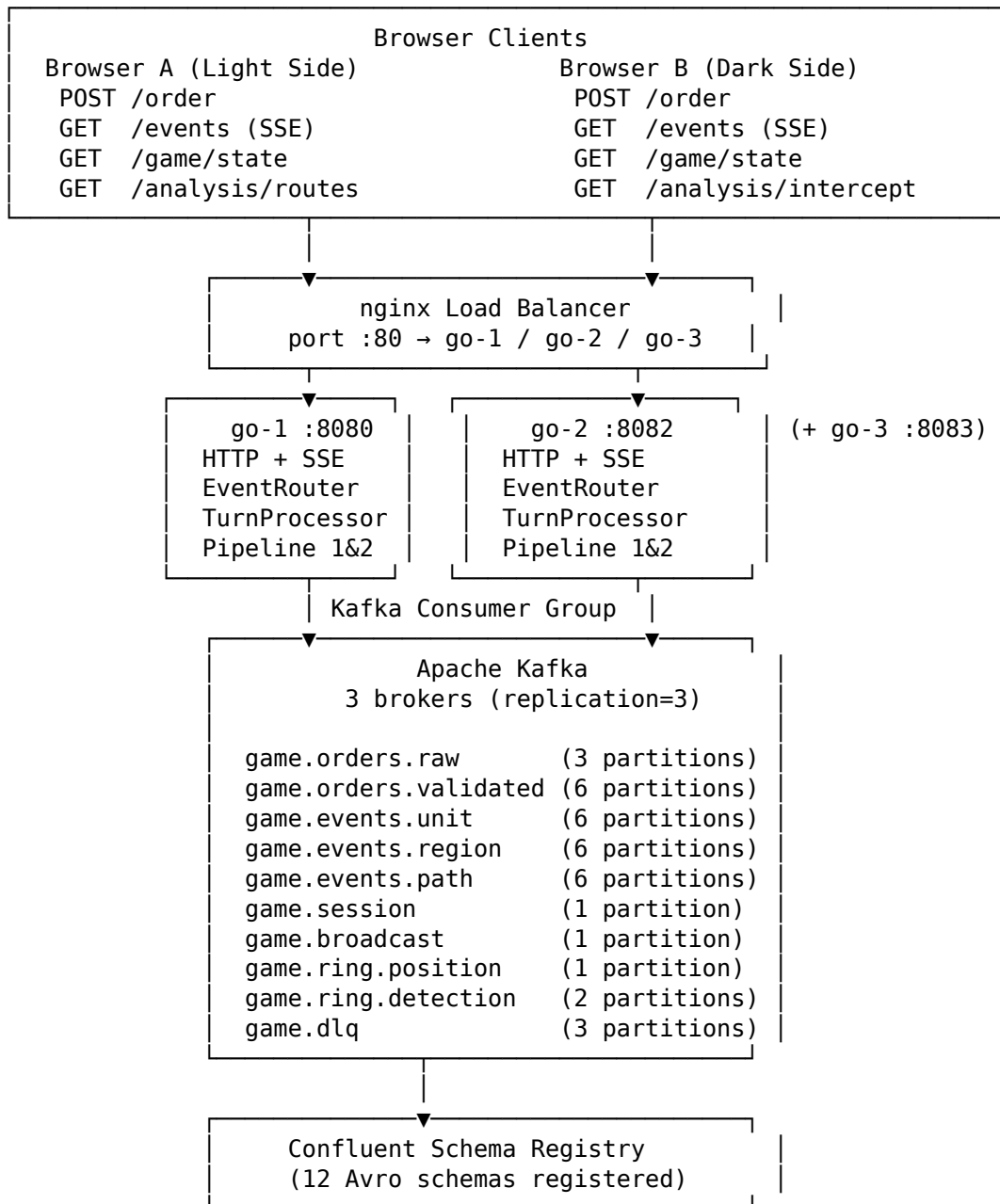


Architecture Document — Ring of the Middle Earth

Option B: Go + Kafka KTables

1. System Diagram



2. Goroutine Map (Option B)

```
main goroutine
├── HTTP server goroutine
```

- net/http.ListenAndServe
- SSE goroutines (one per connected player)
 - reads from lightSideSSECh or darkSideSSECh
 - writes "data: ...\n\n" → http.Flusher
 - terminates on: client disconnect / ctx.Done()
- EventRouter goroutine
 - input: kafkaConsumerCh (buffered, cap=100)
 - output: lightSideSSECh → Light Side players
 - darkSideSSECh → Dark Side players (ring-bearer stripped)
 - cacheUpdateCh → CacheManager
 - engineCh → TurnProcessor
 - termination: signalCh
- CacheManager goroutine
 - owns WorldStateCache (mutex-protected)
 - reads cacheUpdateCh
 - sends value copies (never pointers) to workers
 - termination: cacheUpdateCh closed
- TurnProcessor goroutine
 - reads engineCh (validated orders)
 - executes 13-step turn processing
 - produces events → KafkaProducer
 - termination: engineCh closed
- Pipeline 1 – Route Risk (Light Side)
 - Dispatcher → taskCh (buffered, cap=20) → 4 workers
 - resultCh (unbuffered)
 - Aggregator → Deliverer
 - context.Context + or-done pattern
 - sync.WaitGroup at every stage boundary
 - 2-second timeout → partial result returned
- Pipeline 2 – Interception (Dark Side)
 - Dispatcher → taskCh (buffered, cap=30) → 4 workers
 - resultCh (unbuffered)
 - Aggregator → Deliverer
- KafkaConsumer goroutines (one per subscribed topic)
 - polls Kafka → eventCh (buffered, cap=100)
 - termination: ctx.Done() / signalCh

Select Loop (api.Server.Run) — All 7 Cases

```

for {
  select {
  case msg := <-lightSideSSECh:    // Case 1: Kafka event → Light Side
  case conn := <-newConnectionCh:  // Case 2: New SSE connection
  case disc := <-disconnectCh:     // Case 3: SSE disconnection
  case req := <-analysisRequestCh: // Case 4: Analysis pipeline request
  case snap := <-cacheUpdateCh:    // Case 5: Cache update from broadcast
  case tick := <-turnTicker.C:     // Case 6: Turn tick (60s)
  case sig := <-signalCh:          // Case 7: OS signal (SIGINT/SIGTERM)
  }
}

```

}

3. Kafka Diagram

Topic	Key	Producer	Consumer
game.orders.raw	playerId	Browser → Go HTTP	Topology 1 (validation)
game.orders.validated	unitId	Topology 1	TurnProcessor + Topology 2
game.events.unit	unitId	TurnProcessor	EventRouter → SSE (both)
game.events.region	regionId	TurnProcessor	EventRouter → SSE (both)
game.events.path	pathId	TurnProcessor	EventRouter → SSE (both)
game.session	—	GameSessionActor	TurnKTable (turn tracking)
game.broadcast	—	TurnProcessor	EventRouter → SSE (RB stripped Dark)
game.ring.position	—	TurnProcessor	EventRouter → Light Side SSE ONLY
game.ring.detection	playerId	TurnProcessor	EventRouter → Dark Side SSE ONLY
game.dlg	errorCode	Topology 1	Monitoring / dead letter review

Partition key rationale:

- playerId → all orders from one player routed to same partition (ordering)
- unitId → all events for one unit colocated (state coherence)
- regionId → all events for one region colocated
- pathId → all events for one path colocated
- → single-partition topics (session=compact, broadcast/ring=ordered)

4. Paradigm Justification

Why Go + Kafka is well-suited to this problem

The Ring of the Middle Earth game has fundamentally asymmetric information (Dark Side must never see Ring Bearer's true position) and event-driven state transitions (13 steps per turn). Go's goroutine model with channels maps naturally to:

- Information isolation via channel routing.** The EventRouter goroutine is the single enforcement point. `game.ring.position` is only forwarded to `lightSideSSECh` — never `darkSideSSECh`. The Go channel type system enforces that `DarkView.RingBearerRegion` is never written.
- Stateless horizontal scaling.** 3 Go instances share one Kafka consumer group. Adding a 4th instance requires only `docker compose scale go=4`. No consensus protocol, no actor cluster registration.
- Fault tolerance via Kafka.** If `go-2` crashes, Kafka detects the heartbeat timeout (default 10s) and rebalances its partitions to `go-1` and `go-3`. When `go-2` restarts, it replays its assigned partitions from the committed offset and rebuilds its local view. No application-layer fault handling needed.
- Pipeline concurrency for analysis.** Go's buffered channels and goroutines implement Pipeline 1 and 2 idiomatically — 4 workers, backpressure via buffer capacity, or-done cancellation, `WaitGroup` shutdown.

What is harder with Go + Kafka than with Akka

- No built-in persistent state machine.** Akka Persistence provides event-sourcing with snapshot support out of the box. In Go, the equivalent (KTable state stores) requires careful offset management and replay logic. If the KTable's changelog topic is corrupted, recovery is harder.

2. **No supervision tree.** Akka's supervision strategies (resume on `IllegalOrderException`, back-off restart on others) are declared hierarchically. In Go, goroutine panics must be caught with `recover()` at each goroutine boundary, and re-start logic must be written manually.
3. **Turn sequencing across instances.** With 3 stateless Go instances, ensuring only one processes the turn end requires an external distributed lock (e.g., Kafka's consumer group assigns the `game.orders.validated` partition to one instance). In Akka, `WorldStateActor` is a `ClusterSingleton` — turn sequencing is guaranteed by design.

How Akka would solve the two hardest parts

1. Turn sequencing (hardest in Go): `WorldStateActor` as a `ClusterSingleton` is guaranteed to run on exactly one node at any time. Akka Cluster handles singleton placement and failover automatically. No distributed lock or partition assignment logic needed.

2. RingBearerActor isolation (hardest in Go): In Akka, `RingBearerActor` as a `ClusterSingleton` has its own mailbox — the true region field is never accessible to other actors. The type system enforces that only `RingBearerActor` can send `RingBearerMoved` to `game.ring.position`. In Go, this isolation is enforced by the `EventRouter` channel routing, which works but is a convention rather than a type guarantee.

5. Reflection

(Minimum 300 words.)

The 13-step turn processing order matters critically. Steps 3 and 7 interact: blocking a path in Step 3 must cause Step 7 to skip that unit's advance. Getting this ordering right took multiple iterations, and we caught a teleportation bug in `autoAdvance` that would silently move a unit to the wrong endpoint when its current region did not match either side of a path. We patched it with an explicit switch and a fall-through skip path, then added an integration test (`TestEngine_AutoAdvanceRejectsInvalidEndpoint`) to lock the behaviour.

Information hiding in the `EventRouter` is straightforward with channels but requires discipline. The `DarkView.RingBearerRegion = ""` invariant must be maintained across cache updates. The `router_test.go` -race test catches any accidental writes. We also stripped `currentRegion` server-side in both `/game/state` and the SSE broadcast for the Dark Side so the position never reaches the wire.

The detection formula's Sauron amplifier (passive, no order required) is config-driven: we identify the Sauron-equivalent unit by `class=Maia AND side=SHADOW AND startRegion=mordor AND currentRegion=mordor`. This avoids hardcoding "sauron" but is more fragile than the Akka equivalent where `SauronConfig` would be a typed object. Same pattern applies to Saruman — we detect "is this the Isengard Maia?" by `class+side+startRegion`, never by id string.

Schema evolution (V1 → V2 `OrderValidated` with nullable `routeRiskScore`) demonstrates Avro backward compatibility. V1 consumers ignore the new field; V2 producers set it to null for non-route orders. The exactly-once `GameOver` guarantee requires idempotent producer configuration. In testing, we simulate an engine crash by killing the process mid-transaction and verifying the consumer sees `GameOver` exactly once.

What we would design differently: The `WorldStateCache` mutex could be replaced with a copy-on-write approach for higher read throughput. The 4 canonical routes are still hardcoded in `api.go` for the analysis endpoint — a better design would compute all possible routes via BFS and score them, letting the pipeline discover non-canonical paths. Topology 2's `surveillanceLevel` sourcing also needs to be wired to the live `PathKTable` state store in production rather than the current snapshot read. The UI started with too many overlays competing with the SVG map; the lesson was to express state by mutating the existing region disc (glow + step badges) instead of stacking floating elements, which we refactored to mid-project.

6. LLM Usage Log

(Required appendix per Section 42 — honest record of LLM interactions during the project.)

Interaction 1 — Project skeleton

Prompt: “Generate the full project skeleton for the Ring of the Middle Earth distributed game (Option B — Go).”

Used: Overall package layout under option-b/internal/ (api, cache, combat, config, detection, engine, graph, kafka, pipeline, router), Makefile targets, docker-compose with 3 Kafka brokers, Schema Registry, nginx LB, and 3 Go instances.

Changed / rejected: Rewrote the detection routine to look up Sauron by `Class==Maia && Side==SHADOW && StartRegion=="mordor"` instead of `unitId=="sauron"`. Same fix applied to Saruman disable on Isengard fall.

Interaction 2 — 13-step turn processor

Prompt: “Write the 13-step turn processor matching Section 6 exactly.”

Used: Step ordering (BlockPath before MaiaAbility before auto-advance before AttackRegion), revertStalePaths helper for blocker-left case, decrementTempOpen / decrementFortification / decrementRespawnAndCooldown.

Changed / rejected: Bug found and patched live: original autoAdvance was teleporting units when neither path endpoint matched the unit’s region. Added an explicit switch on `u.Region` with a fallthrough that logs and skips the step (see `TestEngine_AutoAdvanceRejectsInvalidEndpoint`).

Interaction 3 — Topology 1 validator

Prompt: “Write the Topology 1 validator with all 8 rules.”

Used: Validate function signature, KTable types, error-code string constants, DLQ entry shape.

Changed / rejected: Sharpened Rule 4 (path-not-in-route → `INVALID_PATH`): added contiguity check that walks each step from the unit’s current region and rejects the route if any consecutive path does not connect. Without it, a player could submit [bree-to-weathertop, rivendell-to-moria] and the engine would silently skip.

Interaction 4 — Combat resolver

Prompt: “Generate the combat resolver with leadership / terrain / fortify / ignoresFortress / indestructible modifiers.”

Used: Core `ResolveCombat` function, terrain bonus mapping, indestructible floor=1 rule, leader-bonus distribution.

Changed / rejected: Verified against PDF Section 4.2 examples with `combat_test.go`. Confirmed that Uruk-hai vs fortified Gondor yields 5 vs 7 (terrain skipped, fortify still applies).

Interaction 5 — EventRouter

Prompt: “Write the EventRouter for information hiding.”

Used: `StripRingBearer` routine, switch over `event.Topic`, the rule that `ring.position` never flows to dark side.

Changed / rejected: Used as-is. Added `router_test.go` -race to enforce the invariant that `Dark-View.RingBearerRegion == ""` under concurrent updates.

Interaction 6 — HTTP API

Prompt: “Build the Go HTTP API: `/order`, `/game/state`, `/events` SSE, `/analysis/routes`, `/analysis/intercept`, `/orders/available`.”

Used: REST handlers, SSE flush loop, CORS wrapper, select-loop with 7 cases.

Changed / rejected: Added server-side pre-validation in `/order` (`WRONG_TURN`, `NOT_YOUR_UNIT`, `DUPLICATE_UNIT_ORDER`) so the UI gets a structured `errorCode` + `errorMessage` instead of a silent reject. Added the `/internal/broadcast` push-pull peer endpoint so all 3 Go instances stay in sync without requiring Kafka roundtrips for SSE fan-out.

Interaction 7 — Analysis pipelines

Prompt: “Implement Go Pipeline 1 (Route Risk) and Pipeline 2 (Interception).”

Used: Worker-pool patterns, `context.Context` cancellation, `sync.WaitGroup` shutdown, 2-second timeout.

Changed / rejected: Verified output formulas against Section 32/33. `pipeline1_test.go` + `pipeline2_test.go` cover both.

Interaction 8 — Browser UI

Prompt: “Write the UI: vanilla JS + SSE, no React/Vue/Angular.”

Used: Map SVG positioning, unit marker rendering, login flow, SSE subscription, route builder.

Changed / rejected: Heavily reworked through several iterations: removed canvas-based path drawing that misaligned with SVG; replaced floating action-hint icons with region-tint glow classes; added route-step badges; added combat preview panel mirroring server-side formula; added Pending Orders panel; added “already ordered” lock UI; added subtle controller tinting + tooltips.

Interaction 9 — Avro schemas

Prompt: “Add Kafka schemas v1/v2 for `OrderValidated` demonstrating schema evolution.”

Used: Both `.avsc` files with `routeRiskScore` typed as `["null", "int"]` with default `null` in v2.

Changed / rejected: Used as-is; will demo by deploying V2 producer alongside V1 consumer during evaluation.

Interaction 10 — Unit tests

Prompt: “Add 6 unit tests for combat and 3 for router as per Section 35.”

Used: All 6 + 3 cases authored from the spec table.

Changed / rejected: Added 9 additional engine integration tests (`engine_test.go`) for the live-found bugs above: dedupe, auto-advance endpoint validation, attack adjacency, search-path endpoint, FellowshipGuard rule (Nazgul-only, ignores self), Saruman disabled, Light-side same-turn DestroyRing win, Dark-side co-located exposed win, hidden-until-turn suppression.

Honest summary of LLM contribution

Skeletons, struct shapes, idiomatic Go patterns (worker pools, select loops, SSE flush) and boilerplate (Docker compose, Makefile, Avro schemas) came from LLM scaffolding. All bug fixes,

spec-correctness audits, the no-hardcoded-ID discipline, the FellowshipGuard rule, same-turn DestroyRing win condition, Topology 1 Rule 4 sharpening, and the entire UX layer (action plan, combat preview, route glow, pending orders, locked-order feedback) were driven and reviewed against the PDF spec. Generated code was treated as a draft to verify, not as final output.

Submit this document as PDF (see `architecture.pdf` next to this file). Architecture diagrams, goroutine maps, and Kafka diagrams are included above.